DD — Développement du formalisme et de la suite théorique — Order 63

Order 63 — dossier d'exécution

21 septembre 2026

Résumé

Ce document est le dossier principal de l'Order 63. Il décrit uniquement les constructions effectivement codées, les mécanismes reliés causalement au pipeline et les vérifications réellement exécutées. L'ambition de l'architecture est celle d'une IA générale et généraliste, mais aucune universalité n'est déclarée sans preuve. Les résultats négatifs, les frontières de calcul et les branches non promues font partie du résultat scientifique.

1 Statut, héritage et méthode

O63 part de `model_order_62.zip`. La conservation de la base est intentionnelle : les vérificateurs externes, la représentation par empreintes, les structures de déficit, les primitives de contraintes, la VM sandboxée, les optimisations et le gestionnaire de checkpoint ont été repris puis révisés. Les trois refontes structurelles sont : (i) un world model séquentiel à attention, (ii) une navigation RS fondée sur des trajectoires sémantiques et non plus uniquement sur une classification binaire de paires, et (iii) des micro-mémoires locales avec bus typé.

Les modifications sont suivies dans le code sous des versions `O63_*`. Le statut d'un composant est toujours séparé entre « implémenté », « testé localement », « démontré causalement sur un support », et « généralité non établie ».

2 Correspondance avec la théorie des empreintes

Pour une famille de sondes G, l'empreinte observationnelle est

$$I_G(x) = (f(x))_{f\in G}, \qquad x \sim_G y \iff I_G(x) = I_G(y).$$

Le code correspondant se trouve dans `src/order63/geometry.py`, notamment `FingerprintEngine` et `FiberQuotientEngine`. La factorisation est testée par `SufficiencyFactorizationEngine`. La fermeture est opérée par `ObservationClosureEngine` ; la comparaison des partitions permet de vérifier la monotonie du raffinement.

La condition empirique de suffisance utilisée dans les expériences est :

$$I_G(x) = I_G(y) \;\Rightarrow\; A(x) = A(y),$$

sur le support explicitement déclaré. Cela produit une application induite sur les classes lorsque le support satisfait la condition de factorisation. Le système ne transforme pas cette vérification locale en théorème universel.

3 Pipeline O63 effectivement exécuté

Le pipeline réel est :

$$O_t \to P_t \to I_t \to D_t \to C_t \to W_t \to H_t \to \Pi_t \to RS_t \to S_t \to M_t \to Meta_t \to a_t \to V_t \to O_{t+1}.$$

— P_t : encodage perceptif multimodal générique dans `perception.py`.
— I_t : probes, empreintes, fibres, fermeture et géométrie dans `representation.py` et `geometry.py`.
— D_t : vecteur de déficit dans `agent.py:93-118`.
— C_t : contraintes auto-construites dans `constraints.py`.
— W_t : world model sémantique dans `semantic_world_model.py`.
— H_t, Π_t : hypothèses et imagination dans `imagination.py`.
— RS_t : navigateur sémantique dans `rs_navigation.py`.
— S_t : synthèse de programmes et VM dans `synthesis.py`, `program_vm.py`.
— M_t : micro-mémoires locales et bus dans `micro_memory.py`.
— $Meta_t$: méta-contrôle dans `meta.py`.
— V_t : vérification indépendante dans `verification.py` et les environnements externes.

Chaque composant majeur intervient dans la trace, et les interventions causales permettent d'en neutraliser le signal sans supprimer son existence architecturale.

4 World model sémantique

Le nouveau world model utilise un véritable encodeur Transformer séquentiel. Les observations sont canonisées puis tokenisées sous forme d'octets, sans vocabulaire métier prédéfini. Pour une séquence d'entrée $X_{\leq t}$:

$$H_t = \text{Transformer}_\theta(X_{\leq t}), \qquad z_t = \text{Pool}_\theta(H_t).$$

L'implémentation concrète est `SemanticTransformerEncoder` dans `semantic_world_model.py:36-85` avec 96 dimensions internes, 3 couches, 4 têtes et un latent de 128 dimensions. Ces valeurs sont des paramètres d'ingénierie de l'expérience et non des limites théoriques.

La dynamique apprend une distribution approchée :

$$p_\theta(z_{t+1}, r_t \mid z_t, a_t, c_t),$$

avec un ensemble de trois membres de transition qui produit une moyenne et une variance. L'action n'est plus transformée en vecteur à partir du texte de son nom : son vecteur interne est appris à partir de son effet sémantique observé, via `ActionEffectMemory` et `action_embed`. Pour une transition observée :

$$\Delta z_t = z_{t+1} - z_t,$$

et cette variation devient un signal d'apprentissage de l'effet de l'action.

La prédiction fournit simultanément état latent, état successeur prédit, récompense prédite et incertitude. Ces valeurs alimentent directement l'imagination, le RS, la fusion décisionnelle et la mémoire.

4.1 Pré-entraînement RS du world model

La commande réellement exécutée était :

```
cd /tmp/order63_build/model_order_63
export OMP_NUM_THREADS=2 MKL_NUM_THREADS=2 PYTHONPATH=src
time python scripts/train_semantic_world_model.py --dataset /mnt/data/
    dataset_model_order_62_rs.json --out artifacts --seed 63063 --epochs 1
```

Le corpus contenait 320 problèmes et 1436 transitions. Le split par enregistrement a produit 1162 transitions d'entraînement et 274 de holdout. La représentation utilise 27 IDs d'actions opaques. L'erreur MSE du latent sur holdout était $1.7310954884980316 \times 10^{-4}$. L'entraînement a été arrêté à 1 époque parce que la contrainte de calcul était réelle ; il s'agit d'un résultat de ressources, pas d'une limite théorique.

5 RS comme navigation dans l'espace sémantique

Le RS O63 ne reprend pas le classifieur O62 comme objectif final. Il exploite des triplets :

$$(z_0, z_t, z_c, \mathsf{D}_t) \mapsto (p_t, u_t, d_t),$$

où z_c est un état candidat, p_t une progression prédite, u_t une incertitude et d_t une direction latente apprise. La fonction concrète est `SemanticRSNavigator` dans `rs_navigation.py:12-67`.

Les chaînes de démonstration sont conservées comme séquences :

$$z_0 \to z_1 \to \cdots \to z_T.$$

Le signal d'apprentissage de progression utilise la position temporelle dans la trajectoire et des transitions inversées comme négatifs. Des négatifs inter-trajectoires et des réordonnements temporels sont ajoutés dans `scripts/train_rs.py`.

La commande réellement exécutée était :

```
cd /tmp/order63_build/model_order_63
export OMP_NUM_THREADS=2 MKL_NUM_THREADS=2 PYTHONPATH=src
python scripts/train_rs.py --dataset /mnt/data/dataset_model_order_62_rs.json --world
    artifacts/world_model.pt --out rs_artifacts --seed 63062 --epochs 12
```

La séparation est de 256 enregistrements train et 64 holdout. Elle produit 5571 exemples train et 1487 exemples holdout. La MSE de progression holdout est 0.0819355945.

5.1 Évaluation RS

Les mesures internes du navigateur donnent 0.9018106 de précision sur la tâche paire utilisée pour l'audit. Mais ce chiffre n'est pas considéré comme une preuve de navigation utile. Sur les deux familles complètement holdout de l'évaluation dédiée, `symbolic` reste PASS avec 1.0 contre 1.0, tandis que `routing` passe de PASS à EXHAUSTED avec 2.6 contre 1.2 et davantage de nœuds (20 contre 34). La branche RS n'est donc pas promue par défaut.

La campagne appariée finale sur 18 familles donne une moyenne de récompense de 0.205556 pour la base et 0.180556 avec RS. Les statuts sont respectivement {'EXHAUSTED' : 9, 'FAIL' : 4, 'NOT_VERIFIED' : 3, 'PASS' : 2} et {'EXHAUSTED' : 8, 'NOT_VERIFIED' : 4, 'FAIL' : 4, 'PASS' : 2}. Les résultats famille par famille sont conservés dans `results/compare_rs_hard_o63_final2/`.

6 Micro-mémoires locales

O63 remplace la mémoire globale O62 par douze stores locaux. Pour le module k :

$$M_t^k = (E_t^k, H_t^k, P_t^k, V_t^k, B_t^k).$$

Chaque `LocalMicroMemory` stocke type, contenu, embedding issu du world model, provenance, statut, version, dépendances, horodatage, hits et invalidations. La mémoire n'utilise plus de pseudo-embedding cryptographique comme représentation sémantique. La mémoire est indexée par provenance et rapprochée par produit scalaire des états sémantiques.

Le bus est typé :

$$m_t^{i \to j} = T_{ij}(M_t^i, e_t^i),$$

et les routes sont contrôlées dans `MemoryBus.ALLOWED`. Les transitions vérifiées, effets du world model, résultats RS, hypothèses d'imagination, programmes compilés et décisions sont effectivement écrits dans les stores pertinents. Les résultats d'action sont écrits dans la mémoire de politique et peuvent être relus lors d'une nouvelle décision.

Une intervention mémoire existe dans la fusion décisionnelle. La campagne causale est néanmoins claire : sur le support utilisé, le masque mémoire n'a pas changé la récompense dans cette réplication, donc aucune causalité forte de mémoire n'est revendiquée sur ce support. L'absence d'effet est un résultat d'expérience.

7 Imagination récursive

L'imagination utilise un faisceau multi-profondeur. Pour une branche :

$$\Pi = (z_0, a_1, \ldots, a_T, \hat{z}_T, \hat{D}_T, \hat{r}_T, u_T).$$

Les hypothèses couvrent causalité, épistémique, algorithmique, représentation, falsification et compositions récursives. Chaque expansion du world model consomme effectivement le ledger. Le résultat influence la fusion : le score imaginé entre dans les caractéristiques décisionnelles.

La campagne causale `results/causal_interventions_o63/summary.json` donne par exemple : sans masque, récompense -1 ; avec masque imagination, 0.25, et les actions discrètes changent. Cela constitue une intervention locale sur ce support, pas une preuve universelle de supériorité.

8 Synthèse générale de programmes

Le générateur de programmes est neural pour le classement et combinatoire/génératif pour l'exploration d'AST. La VM reste sandboxée. Les services contrôlés incluent `probe`, `predict`, `imagine`, `verify`, `store`, `retrieve` et `act` sous garde.

Un programme n'est pas promu pour la seule exécution. Il doit être input-sensitive sur le support, survivre au holdout et passer la validation prévue. L'acceptation d'un `verify` constant a été supprimée. Les campagnes `program-synthesis` et `program-repair` restent FAIL dans la campagne dure : la présence réelle d'un synthétiseur ne constitue donc pas une résolution générale.

9 Contraintes auto-construites

Les primitives autorisées sont générales : égalité, inégalité, comparaisons, appartenance et compositions booléennes. Les candidats sont construits à partir des chemins observés. Une contrainte possède une provenance et peut être fragilisée par contre-exemple. O63 a corrigé l'erreur conceptuelle d'O62 qui pouvait considérer la présence d'un compteur de contre-exemples comme condition de promotion.

La promotion exige maintenant un support observable d'au moins trois états et l'absence de contre-exemple connu au moment de la promotion. Cela produit une notion de vérification sur le support déclaré, jamais une loi universelle gratuite.

10 Causalité de la décision

La fusion décisionnelle reçoit 13 caractéristiques, dont récompense prédite, incertitude, imagination, signal neural, programme, RS, conformité aux contraintes, représentation, taille du contexte, nombre de programmes, norme sémantique et valeur mémoire. Elle est entraînée sur les récompenses réellement observées après action.

L'intégration RS n'est plus un petit coefficient fixe additionné au score. Lorsqu'il est activé, le score RS est une caractéristique de la fusion apprise.

11 Optimisations

Les optimisations conservatrices reprises d'O62 comprennent fermeture, partition incrémentale, canonicalisation/cache, A*, dominance, MRV, branch-and-bound et équivalence vectorisée. Les tests de conservation sont PASS.

Le benchmark multi-échelle O63 utilise trois répétitions par taille. Les speedups médians sont :

n	baseline (s)	optimisé (s)	speedup
40	0.002821869	0.000275838	10.2302
80	0.011207229	0.000523494	21.4085
160	0.044539707	0.001022500	43.5596
320	0.113956851	0.002111160	53.9783

Une mesure séparée de fermeture donne 14.0041×. Ces nombres sont des mesures de supports expérimentaux et non une loi asymptotique générale.

12 Tests et campagne chess

La suite logicielle finale est de 27 tests passants. Elle inclut les tests O62 hérités et les tests O63 sur Transformer sémantique, RS trajectoriel, micro-mémoire/bus, checkpoint et intégrité.

La campagne chess finale utilise cinq positions avec actions opaques. Les cinq épisodes exécutés après correction de statut se terminent par `NOT_VERIFIED`, avec 4544 évaluations de candidats, 176 branches d'imagination et aucune action externe exécutée. Le vérificateur externe rejette la proposition comme illégale dans l'état courant ; aucun succès chess n'est revendiqué.

13 OOD

Trois seeds OOD donnent tous `EXHAUSTED`, avec récompenses respectives -0.75, 0.50 et -2.00, 8 étapes, 48 nœuds et 128 branches imaginées. La variété des récompenses confirme que l'agent a un comportement effectif mais ne démontre pas une résolution complète OOD.

14 Checkpoints

`FullStateCheckpointManager` sérialise poids, optimiseurs, RNG, trajectoire, probes, contraintes, programmes, mémoires locales, bus, fusion, RS et autres états internes. `run_episode` déclenche désormais automatiquement un checkpoint durable selon `checkpoint_interval`. Les checkpoints OOD sont des artefacts réels dans `results/ood_opaque/ck_*`.

15 Anti-triche et généralité

Le cœur O63 ne contient pas de dispatch vers les noms des benchmarks ou des positions chess. Les noms d'actions sont rendus opaques dans les expériences. Le dataset RS est utilisé seulement pendant le pré-entraînement dédié ; l'évaluation de la release finale n'en a pas besoin.

Le terme « généraliste » désigne ici l'absence de solveur spécialisé et la variété des domaines testés. La campagne ne constitue pas, à elle seule, une preuve d'AGI universelle.

16 Matrice O62 -> O63

O62	correction O63	preuve
latent MLP court	Transformer sémantique + dynamique ensemble	test sémantique + manifest
RS binaire de paires	progression trajectorielle	manifest RS + holdout
mémoire globale/hash	stores locaux + bus typé	tests mémoire
action hashée	effet appris sous IDs opaques	27 actions pré-entraînement
imagination limitée	faisceau récursif multi-profondeur	traces et intervention

synthèse faible	validation train/holdout + services	tests VM + campagne dure
contraintes promotion trop permissive	contre-exemple disqualifiant	test de régression
status ambigu après rejet	NOT_VERIFIED distinct	campagne chess post-correction
optimisation micro seulement	benchmark multi-tailles	optimization_scaling
checkpoint manuel	checkpoint automatique	artefacts OOD

17 Conclusion

O63 est une reconstruction expérimentale nettement plus profonde qu'O62 : le cœur dispose d'un espace de travail neural séquentiel, d'une dynamique apprise, d'une navigation RS trajectorielle, de mémoires locales et d'une imagination récursive effectivement branchée dans la décision. Les résultats ne justifient toutefois pas de déclarer une intelligence universelle : la branche RS n'est pas promue par défaut, plusieurs familles restent difficiles, et chess n'est pas résolu sous le protocole testé. Ces faits sont conservés comme résultats, non transformés en suppositions théoriques.